

Conception et développement d'IHM

Cours S6: Révision, bonnes pratiques, et notions avancées

Contenu

- Comment réfléchir en React ?
- Déploiement desktop
- Frameworks React
- Refs
- Manipuler le DOM avec Refs
- Redux
- Révision générale
- Parties incluses dans l'examen

Comment réfléchir en React ?

Comment réfléchir en React ?

React peut changer votre façon de concevoir les applications que vous créez. Lorsque vous construisez une interface utilisateur avec React, vous la **décomposez** d'abord en éléments appelés **composants**. Ensuite, vous décrivez les **différents états visuels** de chacun de vos composants. Enfin, vous **connecterez** vos composants entre eux de manière à ce que les données les traversent.

Pour mettre en œuvre une interface utilisateur dans React, vous suivrez généralement les cinq étapes suivantes:

Étape 1 : Décomposer l'interface utilisateur en une hiérarchie de composants

Commencez par dessiner des boîtes autour de chaque composant et sous-composant de la maquette et nommez-les. Vous pouvez envisager de diviser une conception en composants de différentes manières :

Programmation : utilisez les mêmes techniques pour décider si vous devez créer une nouvelle fonction ou un nouvel objet. L'une de ces techniques est le principe de la responsabilité unique, c'est-à-dire qu'un composant ne devrait idéalement faire qu'une seule chose. S'il finit par s'étendre, il doit être décomposé en sous-composants plus petits.

CSS : considérez ce pour quoi vous créeriez des sélecteurs de classe. (Toutefois, les composants sont un peu moins granulaires).

Conception : réfléchissez à la manière dont vous organiseriez les couches de la conception.

Comment réfléchir en React ?

Étape 2 : Construire une version statique dans React

Maintenant que vous avez votre hiérarchie de composants, il est temps d'implémenter votre application. L'approche la plus simple consiste à **construire une version qui affiche l'interface utilisateur à partir de votre modèle de données sans ajouter d'interactivité**. Il est souvent plus facile de construire d'abord la version statique et d'ajouter l'interactivité plus tard.

Pour construire une version statique de votre application, vous devrez construire des composants qui réutilisent d'autres composants et transmettent des données à l'aide de « props ».

Vous pouvez construire « **top down** » en commençant par construire les composants situés plus haut dans la hiérarchie ou « **bottom up** » en travaillant à partir des composants situés plus bas. Dans les applications les plus simples, il est généralement plus facile d'aller de haut en bas, et dans les projets plus importants, il est plus facile d'aller de bas en haut.

Étape 3 : Trouver la représentation minimale mais complète de l'état de l'interface utilisateur

Pour rendre l'interface utilisateur interactive, vous devez permettre aux utilisateurs de modifier votre modèle de données sous-jacent. Pour ce faire, vous utiliserez les états « state ». Le principe le plus important pour structurer l'état est de le garder DRY (Don't Repeat Yourself). **Déterminez la représentation minimale absolue de l'état** dont votre application a besoin et calculez tout le reste à la demande. Par exemple, si vous construisez une liste de courses, vous pouvez stocker les articles sous forme de tableau dans l'état. Si vous souhaitez également afficher le nombre d'articles dans la liste, ne stockez pas le nombre d'articles comme une autre valeur d'état - à la place, lisez la longueur de votre tableau.

Comment réfléchir en React ?

Étape 4 : Déterminer ou mettre l'état « state »

Après avoir identifié les données d'état minimales de votre application, vous devez identifier le composant responsable de la modification de cet état, ou propriétaire de l'état. Rappelez-vous : **React utilise un flux de données unidirectionnel, transmettant les données vers le bas de la hiérarchie des composants, du composant parent au composant enfant.** Il se peut que l'on ne sache pas immédiatement quel composant doit posséder quel état. Cela peut s'avérer difficile si vous ne connaissez pas ce concept, mais vous pouvez le découvrir en suivant ces étapes, pour chaque élément d'état de votre application :

1. Identifiez chaque élément qui effectue un affichage en fonction de cet état.
2. Trouvez le composant parent commun le plus proche, c'est-à-dire un composant situé au-dessus de tous les autres dans la hiérarchie.
3. Décidez de l'emplacement de l'état :
 1. Souvent, vous pouvez placer l'état directement dans leur parent commun.
 2. Vous pouvez également placer l'état dans un composant situé au-dessus de leur parent commun.
 3. Si vous ne parvenez pas à trouver un composant dans lequel il est logique de placer l'état, créez un nouveau composant uniquement destiné à contenir l'état et ajoutez-le quelque part dans la hiérarchie, au-dessus du composant parent commun.

Étape 5 : Ajouter le flux de données inverse

L'états « states » circulent vers le bas de la hiérarchie. Mais pour modifier l'état en fonction des entrées de l'utilisateur, **vous devrez prendre en charge le flux de données dans l'autre sens.**

Déploiement desktop

[React Native for Windows + macOS](#) est une solution **officielle** de **MicroSoft** qui apporte la prise en charge de React Native pour **Windows** et **macOS**. Grâce à cela, vos applications React/React Native peuvent être converties en applications **Windows natives** pour tous les appareils pris en charge par Windows 10 et les versions ultérieures, y compris les **PC**, les **tablettes**, la **Xbox**, les appareils de **réalité mixte**, etc. Ainsi que les écosystèmes d'ordinateurs de bureau et d'ordinateurs portables **macOS**.

X Follow @ReactNativeMSFT

React Native for Windows + macOS



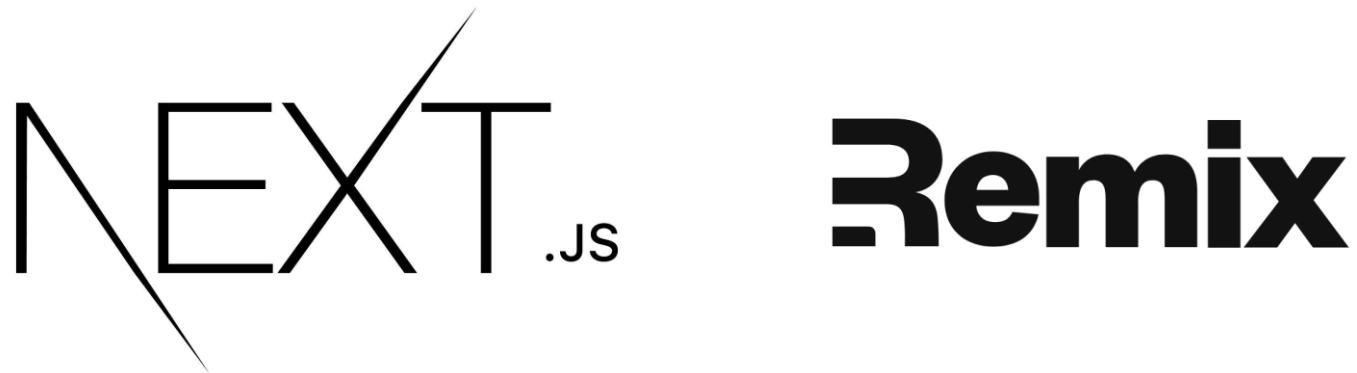
Bring your React Native apps to some of the most powerful devices out there

Frameworks React

Vous pouvez utiliser React sans framework, par contre, la plupart des applications et des sites finissent par trouver des solutions à des problèmes courants tels que le découpage du code, le routage, la récupération de données et la génération de HTML. Ces problèmes sont communs à toutes les bibliothèques d'interface utilisateur, et pas seulement à React.

En commençant par un framework, vous pouvez démarrer avec React rapidement et éviter de construire toutes ces fonctionnalités par la suite. Les deux frameworks recommandés par l'équipe de React sont :

- [Next.js](#) : est un framework React full-stack. Il est polyvalent et vous permet de créer des applications React de toute taille, d'un blog essentiellement statique à une application dynamique complexe.
- [Remix](#) : est un framework React full-stack comme Next.js créé par la même équipe créatrice du package React Router



Refs

Lorsque vous souhaitez qu'un composant se « souviene » d'une information, mais que cette information ne déclenche pas de nouveaux rendus, vous pouvez utiliser une **ref**.

```
import { useRef } from 'react';

export default function Counter() {
  let ref = useRef(0);

  function handleClick() {
    ref.current = ref.current + 1;
    alert('You clicked ' + ref.current + ' times!');
  }

  return (
    <button onClick={handleClick}>
      Click me!
    </button>
  );
}
```

Voici les deux cas d'utilisation les plus courants des refs:

- Stockage et manipulation d'éléments DOM
- Stockage d'autres objets qui ne sont pas nécessaires au calcul du JSX

Manipuler le DOM avec Refs

React met automatiquement à jour le DOM pour qu'il corresponde à votre rendu, de sorte que vos composants n'auront pas souvent besoin de le manipuler. Cependant, il peut arriver que vous ayez besoin d'accéder aux éléments du DOM gérés par React - par exemple, pour focaliser sur une zone de saisie, faire défiler la page automatiquement, etc. React ne permet pas de faire ceci par défaut. Dans ce cas, vous auriez besoin d'utiliser les Refs.

```
import { useRef } from 'react';

export default function Form() {
  const inputRef = useRef(null);

  function handleClick() {
    inputRef.current.focus();
  }

  return (
    <>
      <input ref={inputRef} />
      <button onClick={handleClick}>
        Focus the input
      </button>
    </>
  );
}
```

Redux

Redux est une **bibliothèque de gestion de l'état « state »** des applications JavaScript, souvent utilisée avec React. Elle conserve l'ensemble de l'état de l'application dans un **seul emplacement « store »** et fournit un moyen prévisible de mettre à jour cet état à l'aide d'actions et de réducteurs « reducers ». Les actions décrivent ce qui s'est passé, les réducteurs mettent à jour l'état en fonction de ces actions, et l'immutabilité garantit la prévisibilité. Il est utile pour construire des applications complexes.



Révision

Parties incluses dans l'examen

■ Questions de cours (10 pts)

- Développement d'IHM multiplateformes
- ES6
- L'écosystème de React
- Les notions de base de React: state, props, JSX, refs, ...
- Les formulaires
- Routage
- Les bases de React Native
- La navigation dans React Native

■ Exercice React (10 pts)

- Consommation d'API REST